The Curious Case of Neural Text Degeneration

Holtzman, Buys, Du, Forbes and Choi (ICLR 2020), arXiv:1904.09751

Paper summary for the CS-781 capstone

The paper at a glance

Citation

A. Holtzman, J. Buys, L. Du, M. Forbes, Y. Choi. *The Curious Case of Neural Text Degeneration*. ICLR 2020. University of Washington, Allen Institute for AI, University of Cape Town.

One-sentence summary

Maximising likelihood (greedy, beam search) makes bland, repetitive text, and pure sampling makes gibberish. **Nucleus (top-p) sampling** samples only from the smallest set of tokens holding probability mass p.

Why it matters for our capstone

Phase 1 generates 128-word continuations; decoding directly affects BERTScore. Our `generation.py` implements top-p, top-k, temperature and a repetition penalty.

Where it appears in our slides

Main deck: "Step 05: decoding exactly 128 words". Glossary: "Decoding" section.

Roadmap

1. The problem
2. The basic idea: nucleus sampling
3. Experiments
4. Assessment and outlook

Good models, bad text

Language models trained by maximum likelihood are excellent, yet:

- **Maximisation-based decoding** (greedy, beam search) gives **degenerate** text: bland, generic, and stuck in loops.
- **Pure sampling** from the full distribution gives **incoherent** text that drifts off topic.

With GPT-2 Large and beam widths of 64 or more, the model often prefers to *stop immediately* after the context.

Question: how should we decode *open-ended* text such as stories, or our legal continuations?

The paper's Figure 1, abridged

Context: scientists discover unicorns in the Andes that speak English.

Beam search, $b = 32$: "... researchers from the Universidad Nacional Autónoma de México (UNAM/Universidad Nacional Autónoma de México/Universidad Nacional Autónoma de México/..."

Pure sampling: "They were cattle called Bolivian Cavalleros; ... they speak huge, beautiful, paradisiacal Bolivian linguistic thing."

Open-ended vs directed generation

Directed generation

The output is a tightly constrained transform of the input: translation, data-to-text, often summarisation. Beam search works well enough because the input limits what can be said.

Open-ended generation

Story writing and **text continuation**: many different continuations are acceptable. This freedom is where maximisation fails.

Where is our Phase 1?

In between. Continuing a 512-word judgment is open-ended, but the score compares against *one* hidden true continuation. So the most human-like diversity is not automatically the highest-scoring choice. Tune on the dev set.

Why maximisation fails: human text is not the most probable text

- Per-token probability of **human** text varies a lot: it often picks surprising, informative words.
- Beam-search text stays in a *uniformly* high-probability zone.
- **Repetition is self-reinforcing**: each time a phrase repeats, the model gives it *higher* probability to repeat again. This held for most phrases the authors tested.

The authors' conjecture: people avoid stating the obvious (Grice's maxims), so making every word as predictable as possible is *unlike* human language. Bigger models trained per-word will not fix this alone.

Easy example: the loop

"The court held that the appeal is dismissed. The court held that the appeal is dismissed. The court held..."

After one repeat, "The court held" becomes even more likely, because the context now *contains* that pattern. Greedy decoding cannot escape.

Why pure sampling fails: the unreliable tail

- The vocabulary has tens of thousands of tokens.
- Each tail token has a tiny probability, but **together** the tail can hold a lot of mass.
- So pure sampling picks a strange token surprisingly often.
- One bad token changes the context, and the model "confuses itself" from then on.

Pure-sampling text has *higher* perplexity than human text (22.73 vs 12.38): it is less likely than real text.

Easy arithmetic

Suppose 30,000 tail tokens each have probability 0.00001. Together they hold $30{,}000 \times 0.00001 = 0.30$.

So about **3 steps in 10** would pick a tail token. Over a 180-token continuation, that is roughly 54 odd tokens: plenty to derail a legal paragraph.

Nucleus (top-p) sampling

At each step, take the **smallest set** of tokens $V^{(p)}$ whose total probability is at least p:

$$\sum_{x \in V^{(p)}} P(x \mid x_{1:i-1}) \;\geq\; p$$

Renormalise inside the set and sample:

$$P'(x) = \begin{cases} P(x)/p' & x \in V^{(p)} \\ 0 & \text{otherwise} \end{cases}, \quad p' = \sum_{x \in V^{(p)}} P(x)$$

The **nucleus** usually holds between one and about a thousand candidates. Its size *adapts* to how confident the model is.

Compared with the alternatives

- **Top-k**: always keep exactly k tokens. Truncates in the wrong place when the distribution is very flat or very peaked.
- **Temperature** $t < 1$: sharpens the distribution and shrinks the tail, but also cuts diversity and increases repetition.
- **Top-p**: cuts the tail wherever it starts, and keeps as many good options as there really are.

Easy worked example: peaked vs flat contexts

Peaked: "The appeal is hereby ___"

dismissed 0.80, allowed 0.10, disposed 0.05, withdrawn 0.02, 30,000 others 0.03.

Top-p, $p = 0.9$: $0.80 + 0.10 = 0.90$, so **2 tokens**.
Top-k, $k = 50$: 50 tokens, including 46 from the junk tail, all boosted by renormalisation.

Flat: "The petitioner is a ___"

resident 0.06, company 0.05, farmer 0.05, trader 0.04, teacher 0.04, . . . , many plausible nouns each near 0.01–0.03.

Top-p, $p = 0.9$: perhaps 60 plausible nouns.
Top-k, $k = 3$: only 3, so the text becomes bland and repetitive.

No single k suits both contexts. A single p does, because the nucleus grows and shrinks with the model's confidence. (Probabilities are illustrative.)

Setup and what is measured

- **Model**: GPT-2 Large, trained on WebText (40 GB of web pages).
- **Task**: 5,000 continuations of up to 200 tokens, each conditioned on the first paragraph (1–40 tokens) of a held-out WebText document.
- **Principle**: good decoding should produce text whose statistics are *close to human text*, not as probable as possible.

Measures

- **Perplexity** of the generated text under the model. Closer to human is better.
- **Self-BLEU**: BLEU of each generation against the others. Lower means more diverse.
- **Zipf coefficient**: shape of the word-frequency curve.
- **Repetition %**: generations ending in a loop.
- **HUSE**: combines human typicality ratings (20 annotators $\times$ 200 texts) with model likelihood.

Main results

Method	PPL	S-BLEU4	Zipf	Rep.%	HUSE
Human	**12.38**	**0.31**	**0.93**	**0.28**	–
Greedy	1.50	0.50	1.00	73.66	–
Beam, $b = 16$	1.48	0.44	0.94	28.94	–
Stochastic beam	19.20	0.28	0.91	0.32	–
Pure sampling	22.73	0.28	0.93	0.22	0.67
Sampling, $t = 0.9$	10.25	0.35	0.96	0.66	0.79
Top-$k = 40$	6.88	0.39	0.96	0.78	0.19
Top-$k = 640$	13.82	0.32	0.96	0.28	0.94
Top-$k = 40$, $t = 0.7$	3.48	0.44	1.00	8.86	0.08
Nucleus $p = 0.95$	**13.13**	**0.32**	**0.95**	0.36	**0.97**

Reading the table

- Greedy repeats in **74%** of texts; beam in 29%.
- Their perplexity (about 1.5) is far *below* human (12.4): too predictable.
- Nucleus is closest to human on perplexity, diversity and word frequencies, and gets the best HUSE (0.97).
- The popular top-$k = 40$, $t = 0.7$ recipe repeats in 9% of texts.

Diversity and repetition across settings

Self-BLEU (diversity):

- Common settings $t \in [0.5, 1]$ and $k \in [1, 100]$ make generations too similar to each other.
- The usual range $p \in [0.9, 1)$ matches human diversity.
- Top-k and temperature need extreme values to match, and then coherence suffers.

Repetition: every stochastic method repeats when its knob is set too low, because it then behaves like greedy search. Temperatures below 0.9 sharply increase repetition.

The paper's qualitative example

Context: whales stranded on the West Australian coast.

- **Beam**: repeats one sentence about "more than 50 per cent".
- **Pure sampling**: invents a bird species, "umidauda".
- **Top-k**: veers into an unrelated event.
- **Nucleus**: stays on topic, though it drifts from whales to birds.

Not perfect, but the best of the group.

What this means for Phase 1

Our defaults: $t = 0.8$, top-$k = 50$, top-$p = 0.95$, repetition penalty 1.1, EOS banned.

- Top-p does the main tail cutting; top-$k = 50$ is a safety cap.
- The paper warns that $t < 0.9$ plus small k increases repetition. The repetition penalty helps counter this.
- Greedy decoding is a useful *baseline* to report, since the paper predicts loops.

Experiments worth running

- Greedy vs beam vs top-k vs top-p on dev BERTScore.
- Sweep $p \in \{0.8, 0.9, 0.95, 1.0\}$ and $t \in \{0.7, 0.8, 1.0\}$.
- Measure repetition rate and self-BLEU of your continuations, as in the paper.
- Check whether BERTScore prefers *safer* (lower-entropy) text than human-like text: an important analysis point.

Strengths and limitations

Strengths

- Very simple; a few lines of code; works with any language model.
- Adapts the candidate set to model confidence at every step.
- Explains *why* beam search and pure sampling fail, with measurements.
- Evaluates quality and diversity together (HUSE).
- Became the default decoding method for open-ended generation.

Limitations

- p is still a hand-set hyperparameter.
- A purely probability-based cut; it knows nothing about meaning or facts.
- Studied with one model family (GPT-2) and web text.
- HUSE needed a workaround (interpolating with the full distribution) because truncation gives zero probability to many tokens.
- Human-likeness is not the goal of every task; directed tasks may prefer beam search.

Future research directions (1): from the paper and after

Direction	Details
Characterise the confidence region dynamically *(paper)*	Learn where to truncate, rather than using a fixed p
Semantic utility in decoding *(paper)*	Guide decoding by meaning, not only token probability
Combine training- and inference-time fixes *(paper)*	E.g. unlikelihood training (Welleck et al., 2020) penalises repeated tokens during training
Entropy-aware truncation	Locally typical sampling (Meister et al., 2023); η- and ϵ-sampling (Hewitt et al., 2022)
Perplexity-targeted decoding	Mirostat (Basu et al., 2021) keeps the generated text's surprise near a target
Contrast-based decoding	Contrastive search (Su et al., 2022); contrastive decoding with a weaker "amateur" model (Li et al., 2023)
Scale-aware truncation	Min-p sampling (Nguyen et al., 2024) sets the cut relative to the top token's probability

Future research directions (2): open questions

Open in general

- Why do models give repetition high probability? Is it the training objective or the data?
- Decoding for long documents: keeping topic and facts over thousands of tokens.
- Optimising decoding for a downstream metric without hurting human quality.
- Evaluating diversity and quality together, cheaply.

Open for legal NLP

- Legal text is formulaic, so repetition of fixed phrases is *correct*. How do we penalise loops without penalising legal formulae?
- Constrained decoding that keeps party names, section numbers and outcomes consistent with the prompt.
- Choosing decoding separately for summaries (Phase 6) and continuations (Phase 1).

Summary

- Likelihood-maximising decoding produces bland, repetitive text; repetition reinforces itself.
- Pure sampling falls into the **unreliable tail** and becomes incoherent.
- **Nucleus sampling**: sample only from the smallest set of tokens covering probability p. The set adapts to the model's confidence.
- With GPT-2 Large, $p = 0.95$ comes closest to human text on perplexity (13.13 vs 12.38), diversity and word frequencies, and has the best HUSE score (0.97).
- For us: the core of Phase-1 decoding. Tune p, t and the repetition penalty against dev BERTScore, and report repetition rates.

Reference

Holtzman, A., Buys, J., Du, L., Forbes, M., and Choi, Y. (2020). The Curious Case of Neural Text Degeneration. In *International Conference on Learning Representations (ICLR)*. arXiv:1904.09751.

Other work cited: Hashimoto et al. (2019) HUSE; Welleck et al. (2020) Unlikelihood Training; Basu et al. (2021) Mirostat; Hewitt et al. (2022) Truncation Sampling as Language Model Desmoothing; Su et al. (2022) Contrastive Search; Meister et al. (2023) Locally Typical Sampling; Li et al. (2023) Contrastive Decoding; Nguyen et al. (2024) Min-p Sampling.